K.R. MANGALAM UNIVERSITY

SCHOOL OF ENGINEERING &TECHNOLOGY

Department of Computer Science & Technology

Assignment Number: 3

Unit Title: Sorting Algorithms

Assignment Title: Sorting Algorithms

Course Code: ETCCDS202

Course Name: Data Structures

Faculty: Mrs. Neetu Chauhan

Student Name: Jiya Gupta

Roll Number: 2501730209

Section: A

Programme: B. Tech CSE (Specialization in AI & ML)

1. Table of Recorded Execution Times

Execution Time (milliseconds)

```
Sorting Performance Table (ms)
--------------------------------------------------------------------------------
Size=1000  Type=Random   Insertion=118.722    Merge=5.858      Quick=4.216
Size=1000  Type=Sorted   Insertion=0.337      Merge=3.792      Quick=2.815
Size=1000  Type=Reverse  Insertion=125.571    Merge=4.563      Quick=3.089
Size=5000  Type=Random   Insertion=1296.411   Merge=19.232     Quick=15.25
Size=5000  Type=Sorted   Insertion=1.174      Merge=11.367     Quick=9.561
Size=5000  Type=Reverse  Insertion=1361.441   Merge=8.756      Quick=7.819
Size=10000 Type=Random   Insertion=4440.028   Merge=47.524     Quick=30.808
Size=10000 Type=Sorted   Insertion=2.189      Merge=36.891     Quick=32.158
Size=10000 Type=Reverse  Insertion=12833.885  Merge=42.611     Quick=64.385
```

2. Comparison by Input Type

Random Input

For random input data, Merge Sort and Quick Sort performed faster than Insertion Sort. Insertion Sort required much more time as input size increased.

Sorted Input

Insertion Sort performed best on already sorted data because very few comparisons and shifts were needed. Quick Sort became slower due to poor pivot selection.

Reverse Sorted Input

Insertion Sort performed slowest because maximum shifting was required. Quick Sort also slowed down because partitions became unbalanced.

3. Discussion: $O(n^2)$ vs $O(n \log n)$

Insertion Sort has time complexity of $O(n^2)$, so its execution time increased rapidly for larger datasets. Merge Sort and Quick Sort have average complexity of $O(n \log n)$, therefore they performed much better on large inputs. The practical results matched the theoretical complexities.

4. Special Note for Quick Sort

Quick Sort has average time complexity of O(n log n), but worst-case time complexity of $O(n^2)$. This happens when pivot selection is poor, such as choosing the last element as pivot for already sorted or reverse sorted input. In such cases, performance becomes slower.

5. Stability and Memory Notes

Algorithm	Stability	Memory Usage
Insertion Sort	Stable	In-place
Merge Sort	Stable	Out-of-place (uses extra space)
Quick Sort	Not Stable	Usually In-place

6. Conclusion

Merge Sort and Quick Sort were faster for large datasets. Insertion Sort was efficient only for small or already sorted data. The experiment showed clear differences between $O(n^2)$ and O(n log n) algorithms.